

JAVA SWING & I/O MASTER-SPICKZETTEL

Inklusive vollständiger Import-Schablonen & Verschachtelungsmatrix • 60-Minuten-Klausur-Safe

60-MINUTEN-STRATEGIE FÜR VAN DER KAMP: Unvollständige Klassen oder leere Event-Methoden blockieren die Kompilierung. Schreibe bei Methodenrümpfen sofort Dummies rein (`return null;` oder `return false;`), damit das Projekt *immer* ausführbar bleibt. Nutze konsequent `this`. bei Instanzvariablen und vergleiche Strings niemals mit `==`, sondern mit `.equals()` oder `.equalsIgnoreCase()`.

1. Das Hauptfenster: Klasse Gui (BorderLayout)

Das Hauptfenster bindet alle Sub-Panels ein. Das Standardlayout des JFrame ist das BorderLayout.

```
package de.ostfalia.gui;

// ALLE POTENZIELL MÖGLICHEN IMPORTE FÜR DAS HAUPTFENSTER
import javax.swing.*;      // Für JFrame, JPanel, JButton, JLabel, JTextField, JRadioButton
import java.awt.*;         // Für BorderLayout, FlowLayout, GridLayout, Color
import java.awt.event.*;   // Für(ActionEvent und ActionListener (falls hier direkt genutzt))

public class Gui extends JFrame {
    private GrafikPanel gp = new GrafikPanel();
    private DateiVerwaltung dateiVerwaltung = new DateiVerwaltung();

    // Konstruktor-Injektion zur Verknüpfung der Logikschichten:
    private AnzeigePanel anzeigePanel = new AnzeigePanel(this.dateiVerwaltung);
    private ButtonPanel buttonPanel = new ButtonPanel(this.anzeigePanel, this.dateiVerwaltung);

    public Gui() {
        super("Klausur-Anwendung"); // Fenstertitel im Konstruktor der Oberklasse setzen
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setSize(420, 220); // Dimensionen exakt laut Klausurskizze einstellen!
        this.setLocationRelativeTo(null); // Zentriert das Fenster auf dem Bildschirm

        // Layout explizit setzen (Horizontal Gap: 5px, Vertical Gap: 5px)
        this.setLayout(new BorderLayout(5, 5));

        // Komponenten in die Himmelsrichtungen platzieren
        this.add(this.gp, BorderLayout.WEST);
        this.add(this.anzeigePanel, BorderLayout.CENTER); // Füllt den gesamten Restplatz aus
        this.add(this.buttonPanel, BorderLayout.SOUTH);

        this.setVisible(true); // WICHTIG: IMMER die allerletzte Zeile im Konstruktor!
    }

    public static void main(String[] args) {
        new Gui(); // Startet die Anwendung
    }
}
```

2. Das Formular-Panel: Klasse AnzeigePanel (GridLayout)

Für perfekt ausgerichtete Zeilen (Label + Textfeld untereinander) wird das GridLayout genutzt.

```

package de.ostfalia.gui;

// ALLE POTENZIELL MÖGLICHEN IMPORTE FÜR FORMULAR-PANELS
import javax.swing.*;      // Für JPanel, JLabel, JTextField, JCheckBox, JRadioButton
import java.awt.*;         // Für GridLayout, FlowLayout, Color
import java.awt.event.*;   // Für ActionListener, TextListener (falls Interaktionen hier)

public class AnzeigePanel extends JPanel {
    private JLabel labelName = new JLabel("Name:");
    private JTextField textFieldName = new JTextField(15);
    private JLabel labelDatum = new JLabel("Datum:");
    private JTextField textFieldDatum = new JTextField(10);
    private DateiVerwaltung dateiVerwaltung;

    public AnzeigePanel(DateiVerwaltung dateiVerwaltung) {
        this.dateiVerwaltung = dateiVerwaltung;

        // Parameter: GridLayout(Zeilen, Spalten, H-Gap, V-Gap)
        this.setLayout(new GridLayout(2, 2, 8, 8));

        // WICHTIG: Reihenfolge des Hinzufügens bestimmt die Gitter-Position (Zeile für Zeile)!
        this.add(this.labelName);           // Zeile 1, Spalte 1
        this.add(this.textFieldName);       // Zeile 1, Spalte 2
        this.add(this.labelDatum);          // Zeile 2, Spalte 1
        this.add(this.textFieldDatum);      // Zeile 2, Spalte 2
    }

    public void neuerEintrag() {
        String name = this.textFieldName.getText().trim();
        String datum = this.textFieldDatum.getText().trim();

        if (!name.isEmpty() && !datum.isEmpty()) {
            boolean erfolg = this.dateiVerwaltung.speicherGeburtstag(name, datum);
            if (!erfolg) {
                this.textFieldDatum.setText("Bereits vorhanden!");
                this.textFieldDatum.setForeground(Color.RED); // Setzt Textfarbe auf Rot bei Fehler
            }
        }
    }

    public void loescheFelder() {
        this.textFieldName.setText("");
        this.textFieldDatum.setText("");
        this.textFieldDatum.setForeground(Color.BLACK); // Farbe zurücksetzen
    }
}

```

3. Das Button-Panel mit innerer Klasse ButtonListener (FlowLayout)

Ordnet Knöpfe nacheinander an. Über die innere Klasse wird das Event-Handling realisiert.

```
package de.ostfalia.gui;

// ALLE POTENZIELL MÖGLICHEN IMPORTE FÜR BUTTON-PANELS & ACTIONS
import javax.swing.*;      // Für JPanel, JButton, JLabel
import java.awt.*;         // Für FlowLayout, Color
import java.awt.event.*;   // REIN FÜR DAS EVENT-HANDLING:(ActionEvent, ActionListener)

public class ButtonPanel extends JPanel {
    private JButton buttonHinzu = new JButton("Hinzufügen");
    private JButton buttonZeigen = new JButton("Anzeigen");
    private JLabel labelAnzeige = new JLabel("");

    private AnzeigePanel anzeigePanel;
    private DateiVerwaltung dateiVerwaltung;

    public ButtonPanel(AnzeigePanel anzeigePanel, DateiVerwaltung dateiVerwaltung) {
        this.anzeigePanel = anzeigePanel;
        this.dateiVerwaltung = dateiVerwaltung;

        // FlowLayout ordnet von links nach rechts an (Wunschgröße bleibt erhalten)
        this.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 5));

        this.add(this.buttonHinzu);
        this.add(this.buttonZeigen);
        this.add(this.labelAnzeige);

        // Instanziierung und Zuweisung des Listeners an alle Knöpfe
        ButtonListener listener = new ButtonListener();
        this.buttonHinzu.addActionListener(listener);
        this.buttonZeigen.addActionListener(listener);
    }

    // Innere Klasse für die Klick-Verarbeitung
    private class ButtonListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            String cmd = e.getActionCommand(); // Liest den Beschriftungstext des Buttons aus

            if (cmd.equals("Hinzufügen")) {
                this.anzeigePanel.neuerEintrag();
            } else if (cmd.equals("Anzeigen")) {
                // Beispielaufruf für Anzeigelogik
                this.anzeigePanel.loescheFelder();
            }
        }
    }
}
```

4. Die Logikschicht: Klasse DateiVerwaltung (I/O & Collections)

Verantwortlich für das blockweise, sequentielle Einlesen und Überschreiben von Textdateien.

```

package de.ostfalia.logik;

// ALLE POTENZIELL MÖGLICHEN IMPORTE FÜR DATEI-ZUGRIFF & LISTEN
import java.io.*;           // FileReader, BufferedReader, FileWriter, BufferedWriter, IOException
import java.util.ArrayList; // Für die dynamische Listen-Speicherung
import java.util.Iterator;  // Für sequentielle Durchläufe (z.B. getNextSong())

public class DateiVerwaltung {
    private String pfadListe = "resources/Datenbank.txt";
    private ArrayList liste = new ArrayList<>();

    // Blockweises Einlesen (Beispiel: 2 Zeilen bilden eine funktionale Einheit)
    public void leseListe() {
        // Try-with-resources garantiert das automatische Schließen der Streams!
        try (BufferedReader br = new BufferedReader(new FileReader(this.pfadListe))) {
            String zeile1;
            while ((zeile1 = br.readLine()) != null) {
                String zeile2 = br.readLine(); // Direkt die Folgezeile greifen

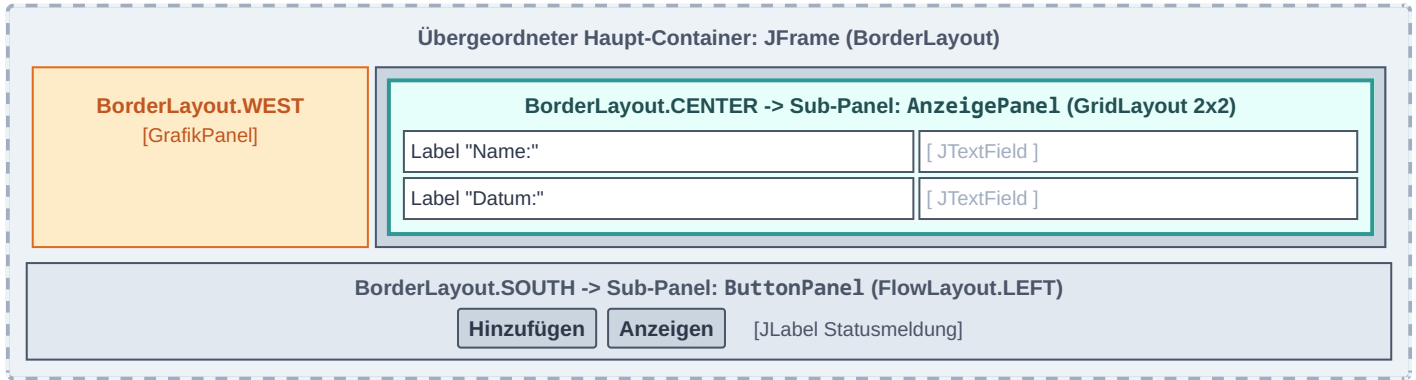
                if (zeile2 != null) {
                    this.liste.add(zeile1.trim() + " ; " + zeile2.trim());
                }
            }
        } catch (IOException e) {
            System.out.println("Fehler beim Einlesen: " + e.getMessage());
        }
    }

    // Komplettes Überschreiben der Datei (false im FileWriter-Konstruktor leert alten Inhalt)
    public void schreibeListe() {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(this.pfadListe, false))) {
            for (String eintrag : this.liste) {
                bw.write(eintrag);
                bw.newLine(); // WICHTIG: Setzt das Zeilenumbruch-Zeichen für die nächste Zeile!
            }
            bw.flush(); // Puffer leeren
        } catch (IOException e) {
            System.out.println("Fehler beim Schreiben: " + e.getMessage());
        }
    }
}

```

5. Visualisierung & Entscheidungsmatrix: Verschachtelte Layouts

Die Skizze unten zeigt, wie die oben deklarierten Klassen nahtlos ineinander verschachtelt werden:



6. Layout-Verhaltensmatrix & Swing-Kniffe

Komponente / Layout	Verhalten im Fenster & Größenänderung	Wichtige Steuerungs-Methoden
BorderLayout.NORTH / SOUTH	Wird auf die volle Breite gestreckt. Die Höhe bleibt exakt auf der Wunschgröße der Elemente.	<code>button.setEnabled(false);</code> Deaktiviert Knöpfe (wichtig für Start/Aufgeben-Logik).
BorderLayout.WEST / EAST	Wird auf die volle Höhe gestreckt. Die Breite bleibt exakt auf der Wunschgröße des Panels.	<code>textField.setEditable(false);</code> Setzt Textfelder auf schreibgeschützt.
BorderLayout.CENTER	Radikale Streckung! Dehnt die Komponente rücksichtslos in Breite UND Höhe aus, um allen Restplatz zu füllen.	<code>label.setForeground(Color.RED);</code> Ändert Schriftfarben (z.B. für "Login erfolgreich!" in Grün).
GridLayout (Formular)	Teilt den Raum in gleich große Quadrate. Alle Zellen werden exakt identisch groß gezogen.	<code>int wert = Integer.parseInt(tf.getText().trim());</code> Sicheres Wandeln von Texteingaben in Zahlen.
FlowLayout (Buttons)	Komponenten behalten ihre absolute Wunschgröße bei. Fließen stur von links nach rechts durch.	<code>int r = (int)(Math.random() * 100) + 1;</code> Generiert eine Klausur-Zufallszahl zwischen 1 und 100.

Vermeide Punktabzug (Java-Programmierrichtlinien nach Van der Kamp):

- Instanzvariablen:** Aufrufe innerhalb der eigenen Klasse konsequent mit `this.` absichern (z.B. `this.add(...)` oder `this.liste`).
- Klassennamen:** Immer in PascalCase verfassen (z.B. `DateiVerwaltung`, `ButtonListener`).
- Variablen/Methoden:** Immer in camelCase verfassen (z.B. `leseListe()`, `textFieldName`).
- Konstanten:** Komplette in GROSSBUCHSTABEN mit Unterstrichen schreiben (z.B. `public static final int BREITE = 400;`).